EMERGENCY CONTACT ALERT SIMULATION

PROJECT REPORT

NAME: PORANDLA SRI CHARANI

REG. NO: 25BEC10101

BRANCH: ECE CORE

COURSE: Introduction to Problem Solving and Programming (CSE1021)

FACULTY : POONKUNTRAN

SLOT: C11+ C12+ C13

DATE: 31 MARCH 2026

1. INTRODUCTION:

Emergencies can happen to anyone, at any time. Whether it is a medical situation, getting lost in an unfamiliar place, or any other crisis, the first instinct is to contact someone who can help. However, under panic and stress, searching through a phone's contact list and calling people one by one wastes precious time.

The Emergency Contact Simulator is a Python command-line application I built to address this problem. It allows a user to store a list of emergency contacts and, with a single input, simulate broadcasting an alert message to all of them at once. While the alerts are simulated (no real SMS is sent), the application demonstrates a practical, working prototype of how such a system could function. The system allows users to store emergency contacts, trigger alerts, and maintain a record of previous alerts.

This project was developed as part of the Python Essentials course to apply foundational programming concepts in solving a real-world inspired problem. It focuses on simplicity, usability, and practical implementation using core Python features.

1.1 Why This Idea?

In real-life situations such as accidents, medical emergencies, or unsafe environments, a person may not have enough time to manually contact multiple people. This inspired the idea of creating a system where a single action can notify all important contacts.

As a beginner in Python, I wanted to build something:

- Useful in real-life scenarios
- Simple yet meaningful
- Aligned with concepts I have learned

This project reflects my interest in combining technology with practical problem-solving.

1.2 Scope of the Project:

The scope of this project includes:

- Managing emergency contacts
- Simulating alert messages
- Logging alert history
- Providing a simple user interface

However, the system is limited to:

- Command-line interaction
- Simulation only (no real message sending)

It serves as a prototype model for understanding how such systems work.

2. PROBLEM STATEMENT:

This is a real observation from my own life.

"How can a system be designed to quickly notify multiple contacts in an emergency using simple logic and minimal user input?"

When something goes wrong, people panic. A structured system that already has your contacts stored and can broadcast an alert in seconds solves that problem. I also wanted the message options to reflect realistic scenarios — being lost, being injured, feeling unwell, or simply confirming you are safe.

This problem maps well to what we studied in the Python Essentials course:

- Dictionaries — to store contacts as name-number pairs
- Lists — to maintain the alert log
- Functions — to separate each feature cleanly
- Loops — to iterate over contacts and display menus
- Conditionals — to handle menu choices and input validation
- Modules — date-time for timestamps, time for simulated delays

3. APPROACH TO SOLVE THE PROBLEM:

The problem was approached by designing a menu-driven command-line application with the following steps:

1. Contact Management
    - Store contacts using a dictionary
    - Allow adding and deleting contacts
2. Alert Mechanism
    - Provide predefined emergency messages
    - Allow custom message input
    - Send alerts to all contacts (simulation)

3. Data Persistence
   a. Store contacts
   b. Store alert logs using text file
4. User Interaction
   - Use a loop-based menu system
   - Accept user inputs for operations

This approach ensures that the system is:

- Simple
- Functional
- Easy to understand
-

4. KEY DECISIONS MADE:

Several important decisions were taken during development:

- Use of CLI instead of GUI
  - Keeps the project simple and focused on logic
- Using Dictionary for Contacts
  - Efficient key-value storage (name → phone number)
- Simulation instead of real API integration
  - Avoids complexity beyond course scope
- Modular Functions
  - Improves readability and maintainability

These decisions helped balance simplicity and functionality.

5. CHALLENGES FACED:

During development, several challenges were encountered:

- Handling file errors when files did not exist
- Managing user input validation
- Ensuring data persistence across runs
- Structuring the program logically
- Avoiding crashes due to invalid inputs

These challenges helped improve debugging and problem-solving skills.

6. WHAT I LEARNED?

6.1 Technical Skills

Through this project, I learned:

- Use of functions for modular programming
- Working with dictionaries and lists
- Implementing file handling
- Using loops and conditionals effectively
- Using modules like:
  - date-time (timestamps)
  - time (simulation delay)

6.2 Problem Solving and Design

- Breaking down a problem into smaller parts
- Designing logical flow before coding
- Thinking about user interaction
- Improving efficiency and usability

6.3 Software Development Process

- Writing code in stages
- Testing features step-by-step
- Debugging errors
- Improving based on feedback

This project gave a basic understanding of how real software is developed.

7. PYTHON CONCEPTS APPLIED:

Concept	Where it appears in my code
`Dictionary`	cntcts — stores contacts as key-value pairs
`List`	lgBk — stores alert history; optns dict for messages
`Functions`	6 separate functions, each doing one job
`for loop`	Iterating over cntcts in list_cntcts() and trigger_alert()
`while loop`	The main app_loop() that keeps the program running
`if / elif / else`	Menu routing, input validation, empty-check logic
`dict.pop(key, None)`	Safe deletion without crashing if key missing
`List slicing lgBk[-3:]`	Showing only the 3 most recent log entries
`try / except`	Catching KeyboardInterrupt (Ctrl+C) gracefully
`f-strings`	Formatting all output messages and the alert report
`import datetime`	Timestamping each alert with the current time
`import time`	time.sleep() for realistic broadcast simulation
`str.strip()`	Cleaning up user input to avoid spaces/empty entries

8. HOW TO RUN THE PROJECT:

- Install Python 3.x or any other python compiler
- Download or clone the project
- Open terminal in project folder
- Run the command: EmergencyContactAlertSimulationproject.py
- Follow on-screen instructions

9. OUTPUTS:

```
======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 1

Contact list

- Father : 8374567363
- Mother : 7346929283
- Brother : 8346596343
- Sister : 9385698477
- Best Friend : 6546678994
```

```
======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 2
Name: Uncle
Phone number: 8658376745
[+] Saved Uncle to contacts.

======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 1

Contact list

- Father : 8374567363
- Mother : 7346929283
- Brother : 8346596343
- Sister : 9385698477
- Best Friend : 6546678994
- Uncle : 8658376745
```

```
======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 3
Who do you want to remove? Uncle
[-] Removed Uncle.

======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 1

Contact list

- Father : 8374567363
- Mother : 7346929283
- Brother : 8346596343
- Sister : 9385698477
- Best Friend : 6546678994
```

```
======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 4

SELECT ALERT TYPE:
1. Emergency, I'm lost, please help
2. Current city is Bhopal
3. Medically I'm not feeling well.
4. I'm okay.
5. injured, help.
6. Write your own..
Selection: 6
Type message: Someone pick me from college
Sender name: Charani

Initiating alert sequence...
Sending to Father..
Sending to Mother..
Sending to Brother..
Sending to Sister..
Sending to Best Friend..

Done! Sent to 5 people

    Report:
    From: Charani
    Msg: "Someone pick me from college"
    Time: 01:00:54
```

```
======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 5

=== RECENT ACTIVITY ===
[01:05:36] Charani: Someone pick me from college
========================
```

```
======= EMERGENCY CONTACT STIMULATOR v1.0 ======
1. My Contacts
2. Add New
3. Delete Contact
4. TRIGGER ALERT
5. History
6. Quit
: 6
Exiting system...
```

10. LIMITATIONS:

- The system is a simulation and does not send real alerts or messages.
- It uses a command-line interface, which may not be very user-friendly.
- Location is manually entered, not tracked in real time.
- No authentication or security features are implemented.
- It has limited validation for inputs like phone numbers.
- No proper integration with external APIs or emergency services.

11. FUTURE IMPROVEMENTS:

This project can be further enhanced in several ways to improve its functionality and real-world applicability:

- Integrating SMS APIs would allow the system to send real alerts instead of simulations, making it more practical.
- Developing a graphical user interface (GUI) would improve usability and make the system more accessible to non-technical users.
- Incorporating automatic location detection using GPS or APIs would make alerts more accurate and efficient.
- Adding authentication and security features would help protect user data and prevent unauthorized access.
- Expanding the system to support multiple users or contact groups would improve its scalability.

Overall, these improvements would help transform the project into a more complete and real-world applicable solution.

12. CONCLUSION:

This project helped me understand how the concepts learned in Python can be applied to solve real-world problems in a simple and meaningful way. Instead of focusing only on individual exercises, I was able to combine multiple concepts such as functions, dictionaries, loops, and file handling into a single working application.

One of the major learnings from this project has been that while creating a program, it is not just about creating a program, but also about creating a program that is not just functional, but also structured, readable, and user-friendly.

I also learned the importance of looking at the problem from the end user's point of view. Handling unexpected inputs, providing adequate feedback, and providing smooth interaction are all very important aspects of creating a reliable program. Although the project is a simulation, it gave me a clear idea of how real emergency systems might be designed and implemented. It also increased my confidence in building complete applications from scratch.

Overall, this project was a valuable learning experience that strengthened my programming fundamentals, improved my problem-solving approach, and motivated me to explore more advanced and practical applications of Python in the future.

13. REFERENCES:

- Python code execution: Spyder (Python 3.13)
- Python official documentation for basic syntax and input/output functions
- Class notes and course materials provided during the module.